
Linux

Sistema Operativo UNIX

- Fue creado en los Laboratorios Bell en Estados Unidos.
- Sucedió al proyecto MULTICS entre Bell y General Electric, el que no tuvo éxito.
- Ken Thompson hizo las primeras experiencias de lo que sería el Unix, en una Digital PDP-7
- Durante el año 1969 Dennis Ritchie (autor del lenguaje C) ayudó a trasladarlo a una PDP-11.

Sistema Operativo UNIX

- Se buscaba la interactividad que no tenían los mainframes, especialmente en la etapa de desarrollo de programas.
- En el año 1971 se generó la segunda versión de UNIX, utililzándose un PDP-11/45, desde aquí pasó a ser multiusuario.
- Thompson había escrito un lenguaje llamado "B" y pretendía escribir el núcleo (Kernel) en este lenguaje.

Sistema Operativo UNIX

- Para esta época Ritchie había creado el "C" como sucesor del "B", por lo que terminan escribiendo el kernel en "C".
- Esto rompía la tradición de escribir los sistemas operativos en lenguaje ensam-blador.
- Desde aquí UNIX y "C" pasaron a quedar estrechamente relacionados.
- Se usa en forma interna en Bell hasta el año 1975.

Sistema Operativo UNIX

- Durante el año 1975 Bell, que estaba inhibida de vender productos de informática, lo distribuye en forma gratuita, con fuentes incluidos a universidades norteamericanas.
- En 1977 se traslada a la primera máquina no fabricada por DEC, una INTERDATA 8/32 de Data General.
- De aquí nace la legendaria facilidad de migración que poseen el UNIX y "C".

Sistema Operativo UNIX



Ken Thompson



Dennis Ritchie

La fuerza propulsora detrás de los Laboratorios de la Bell Computer en New Jersey, con respecto a uno de los sistemas operativos más populares de la historia, estuvo conformada por 2 prominentes investigadores: Dennis Ritchie y Keneth Thompson quienes desarrollaron desde 1969 a 1971 el famoso sistema operativo UNIX.

Ken Thompson es Master en Ingeniería Eléctrica, graduado en la Universidad de Berkeley en California. Es miembro de los Laboratorios Bell, de la National Academy of Sciences, la National Academy of Engineering y la Association for Computing Machinery (ACM).

Dennis Ritchie creó el también famoso Lenguaje C, pero fue Ken Thompson, quien antes desarrolló el Lenguaje B, predecesor e inspirador del Lenguaje C. Asimismo, Thompson reescribió el Kernel de UNIX para perfeccionarlo.

Dennis Ritchie, graduado en Física y Matemáticas Aplicadas en la Universidad de Harvard, todavía es miembro de los Laboratorios Bell y la corporación Lucent Technologies.

Sistema Operativo UNIX

- La Universidad de Berkeley genera su propia distribución llamada BSD, es actualmente una de las dos grandes ramas del Unix moderno. La otra es la derivada de AT&T con su versión V.
- En 1991 se asocian USL de AT&T y Novell Inc. para generar una versión denominada UnixWare, posteriormente Novell compra toda la sociedad y coloca la licencia en manos de la OSF.

Nacimiento del Linux

```
Hello everybody out there using minix -

I'm doing a (free) operating system (just a
hobby, won't be big and professional like
gnu) for 386(486) AT clones.
```

- **En agosto de 1991, el joven estudiante finlandés Linus Torvalds, de la Universidad de Helsinki envió este mensaje al grupo de noticias comp.os.minix para invitar a otros al desarrollo de un “hobby”.**

Nacimiento del Linux

- A partir de aquí se genera una de las revoluciones más interesantes en el mundo de la computación.
- Esto no es por el producto en si, sino por la filosofía de trabajo.
- El desarrollo es verdaderamente abierto, el sistema operativo es gratuito, los fuentes están disponibles y todo el mundo está invitado a participar en el proyecto.

Nacimiento del Linux

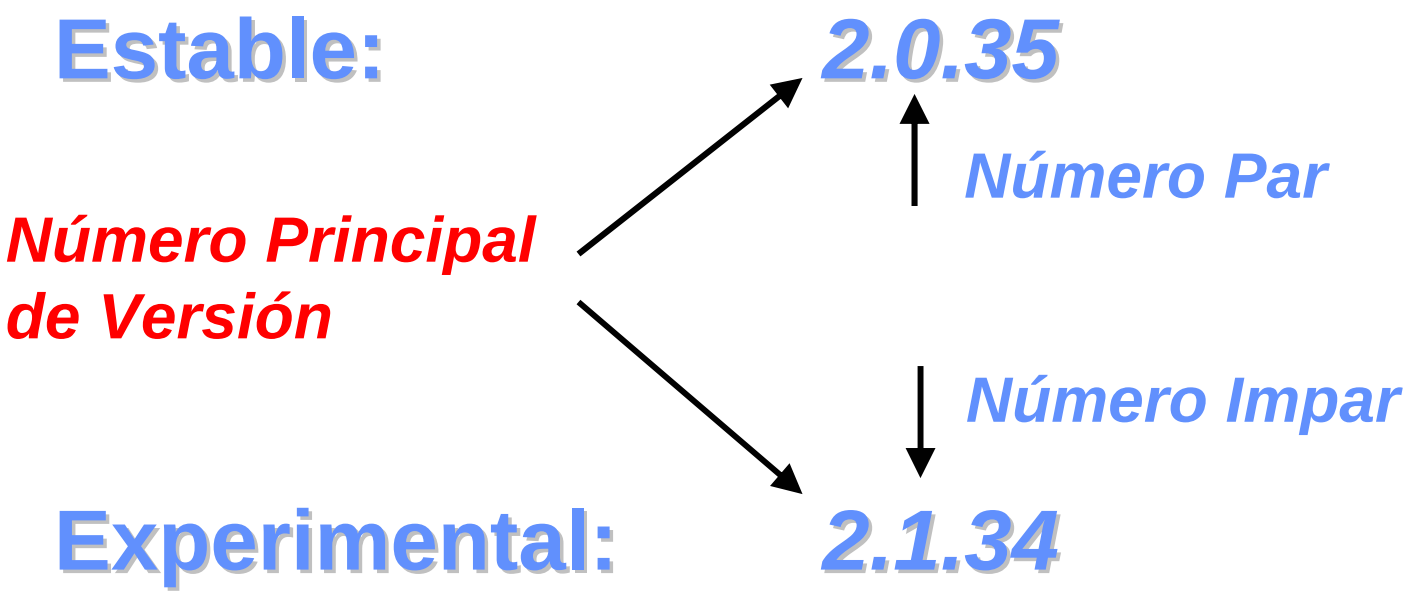
- En sus comienzos fue una herramienta de hackers (en el buen sentido de la palabra).
- El trabajo realizado por Linus Torvalds y el resto de los desarrolladores sobre el kernel.
- Linus mantiene el control de las versiones y comanda el desarrollo general, tarea que ha llevado como actividad paralela a su trabajo habitual después de haber egresado de la Universidad de Helsinki.

Caracteristicas del Linux

- Hay dos líneas de kernel, perfectamente diferenciadas, una estable, probada, lista para entrar en producción y otra en el canal de desarrollo, para quienes quieren aventurarse con nuevas características.
- Aproximadamente cada 15 o 20 días aparece una nueva versión del Kernel, con correcciones y agregados.
- No es imprescindible tener la última versión instalada.

Caracteristicas del Linux

- **Líneas de kernel**



Caracteristicas del Linux

- Desde hace bastante tiempo se empezó a utilizar en ambientes de producción, con excelentes resultados .
- Es proverbial su estabilidad y tolerancia a fallas, aún graves.
- Linux es POSIX compatible, por lo que su estructura es estándar y capaz de competir con los sistemas comerciales en igualdad de condiciones... o con ventajas, por lo inmenso de su “grupo de ingeniería”.

Distribuciones de Linux

- Las distribuciones son las formas de empaquetar el sistema operativo, dándole flexibilidad y facilidad a la instalación.
- Actualmente se provee en CDs y DVDs que tienen su costo, pero lo que se cobra es el trabajo producido para armar la distribución y el medio de soporte.
- Las empresas no se limitan a “guardarlo” convenientemente sino que generan herramientas para facilitar la instalación.

Distribuciones de Linux

Las principales son:

- **Debian**
- **Red Hat / Fedora**
- **SuSE**
- **Slackware**
- **Knoppix**
- **Ubuntu**
- **Mandrake**
- **Mandriva**
- **Gentoo**

Distribuciones de Linux

- También se disponen versiones ISO para grabar CDs o DVDs de las principales distribuciones.
- Existen sitios en la Internet dedicados a almacenar imágenes en formato iso9660.
- La gran división actual se plantea entre distribuciones instalables y las que corren directamente a partir del medio óptico de arranque.

LinuxISO.org

Fresh ISOs, just like mom used to burn.

Site News

Wednesday, May 17th
- openSUSE 10.1 i386 Added to Downloads

Thursday, May 11th
- FreeBSD 6.1 Released!

Friday, April 28th
- Link Updates Have Begun!

Tuesday, April 25th
- Cool Mind Map of Linux Distributions

More Site News...

Helpful Stuff:

About Linux:
- Introduction to Linux

Site hot spots:
- LinuxISO.org Forums
- Submit your Distro

Help with iso images:
- ISO Image FAQ
- How to Burn ISOs
- Verifying ISO Images

About this site:
- State & History
- Banners & Link Buttons
- Donations
- Contact Page

Most popular distributions.

Peanut Linux | arch linux | debian
Fedora PROJECT | FreeBSD | Gentoo
KNOPPIX | Mandrake Linux | SuSE
Red Hat | slackware | TSL
turbolinux. | Vector Linux | Yellow Dog

Don't see your favorite distribution above ?

Use the menus below to view all distributions alphabetically, or to filter by supported architectures.

– By Letter – Go! <-- Sorting method --> – By Architecture – Go!

Or if you still can't find it, try submitting it to us so we can add it.

Help us out:

Currently, linuxiso.org is supported through our advertisers. If you would like to advertise, click here.



Advertise on LINUXISO.ORG

Advertise on LINUXISO.ORG

All images, trademarks and copyrights herein are property of their respective owners. All else Copyright © 2000-2002 Linuxiso.org.

Welcome to Knoppix.net

Knoppix is a Live Linux CD based on Debian GNU/Linux

This website is about Knoppix, a Free and Open Source Live Linux CD. Knoppix is a GNU/Linux distribution that boots and runs completely from cd. It includes recent linux software and desktop environments, with programs such as OpenOffice.org, Abiword, The Gimp, Konqueror, Mozilla, Apache, PHP, MySQL and hundreds of other quality open source programs.

Knoppix.net is a resource for users, developers, and testers of Knoppix. The official website for Knoppix is on Klaus Knopper's website, at knopper.net/knoppix/index-en.html.

If you would like to learn more about Knoppix, you could look at the FAQ's and Documentation, discuss it in the forums, or just try it out! - You can download it (700MB) or buy it from a CD distributor, more info on the download / buy page.

If you have found a bug in Knoppix, you can report it on the Bug tracking site.







Linux

Bug Tracking (download)

RSP for the iSeries

ubuntu

Search Search

Ubuntu | Community | Support | Partners | Wiki

linux for human beings



desktop

- Information
- Download



server

- Information
- Download



community

- Browse/Join
- Developers



support

- Commercial
- Community

ubuntu

- About Ubuntu
- Download
- Shipit - Free CDs
- Donations
- Ubuntu Shop
- Security notices
- Employment
- Do you Ubuntu?

ubuntu 6.10



Ubuntu is a complete Linux-based operating system, freely available with both community and professional support. It is developed by a large community and we invite you to participate too!

The Ubuntu community is built on the ideas enshrined in the Ubuntu Philosophy: that software should be available free of charge, that software tools should be usable by people in their local language and despite any disabilities, and that people should have the freedom to customise and alter their software in whatever way they see fit

ubuntu spotlight

ubuntu LIVE

July 22–24, 2007

Portland, Oregon

CFP Now Open

celated projects

debian

Elija un servidor cerca de usted

Estados Unidos Ir

Acerca de Debian
- Contrato social
- Software libre
- Colaboradores
- Donaciones
- Contacte con nosotros

Noticias
- Noticias semanales
- Eventos

Cómo obtener Debian
- Vendedores de CD
- Imágenes ISO de CD
- Instalación por red
- Preinstalado

Paquetes Debian

Documentación
- Info sobre versiones
- Manual de instalación
- Libros sobre Debian

Soporte
- Debian es internacional
- Información sobre seguridad
- Informes de fallo
- Listas de correo
- Archivos de listas de correo
- Migraciones/Arquitecturas

¿Qué es Debian?

Debian es un sistema operativo (SO) libre, para su computadora. El sistema operativo es el conjunto de programas básicos y utilidades que hacen que funcione su computadora. Debian utiliza el núcleo Linux (el corazón del sistema operativo), pero la mayor parte de las herramientas básicas vienen del Proyecto GNU; de ahí el nombre GNU/Linux.

Debian GNU/Linux ofrece más que un SO puro; viene con unos 15490 paquetes, programas precompilados distribuidos en un formato que hace más fácil la instalación en su computadora.

Lea más...

Primeros pasos

La última versión estable de Debian es la 3.1. La última actualización de esta versión se publicó el 18 de febrero de 2007. Puede leer más sobre las versiones disponibles de Debian.

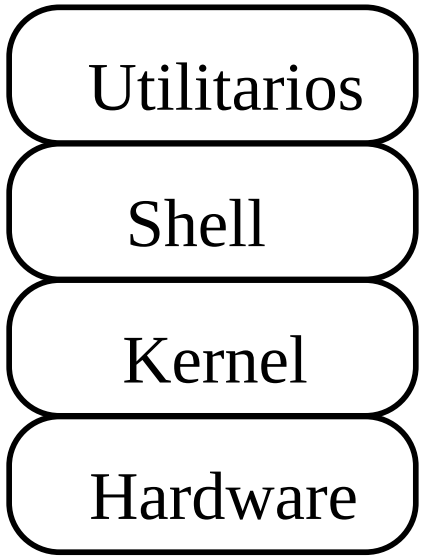
Si desea empezar a usar Debian, puede obtener una copia con sencillez, y seguir las instrucciones de instalación para instalarla.

Si está actualizando a la última versión estable desde una versión anterior, por favor, lea las notas de publicación antes de hacerlo.

Para obtener ayuda sobre el uso o instalación de Debian, vaya a nuestras páginas de soporte y documentación.

Los usuarios que hablen en idiomas que no sean inglés deberían echar un vistazo a nuestra sección internacional.

Arquitectura de Linux



El Kernel se comunica con el hardware. No tiene ninguna vinculación con el usuario. Coordina las funciones internas y administra los recursos del sistema.

El Kernel Linux

- Las dos funciones principales del Kernel son:
  - Administración de Procesos
  - Administración de Dispositivos

- **Administración de Procesos:** Planifica el tiempo de corrida de cada uno y los privilegios que tienen.
- **Administración de Dispositivos:** Supervisa la transmisión de datos entre la memoria principal y los periféricos.

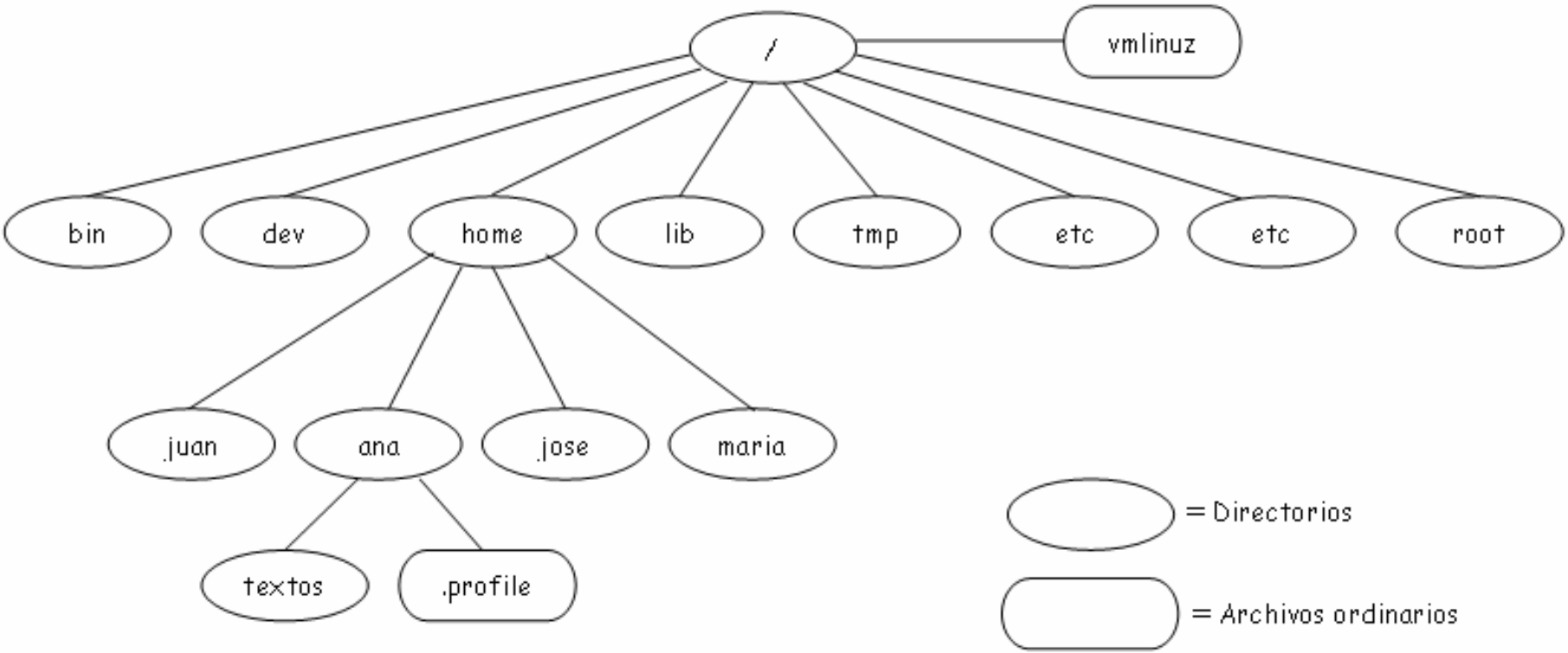
Los Shells en Linux

- Es la cara visible al usuario, interpreta las órdenes que recibe y las transmite al kernel mediante System Calls.
- Hay varios disponibles:
  - sh SHell estándar o de Bourne
  - csh c-shell
  - ksh Korn SHell
  - bash Bourne Again SHell.

Interacción Kernel/Shell

- Si un usuario quiere borrar el archivo carta.txt, en forma gráfica o en una terminal de texto, se ejecuta el comando:
  `rm carta.txt`
- En primer lugar el shell busca el comando `rm` en el sistema de archivos, si lo encuentra le solicita al kernel (mediante una llamada a sistema) que lo aplique sobre el archivo `carta.txt`.

El Sistema de Archivos



Archivos en Linux

- En Unix todos son archivos, sean estos archivos de datos, directorios, impresora, memoria RAM o dispositivos de almacena-miento.
- Esta simplificación es una las causas principales del éxito de UNIX.
- Los archivos en Unix no poseen ninguna estructura. Esta es administrada por las aplicaciones.

Tipos de Archivos en Linux

- Son los siguientes:
  - Archivos Ordinarios
  - Archivos de Directorio
  - Archivos de Dispositivos

Archivos Ordinarios: Son simplemente una coleccción de bytes de 8 bits, documentos, programas fuente, ejecutables, etc. El contenido es interpretado por los programas de aplicación.

Archivos Ordinarios

Tienen los siguientes atributos:

- **Un nombre (no necesariamente único)**
- **Un número único dentro del file system, llamado número de i-nodo.**
- **Un tamaño en bytes.**
- **Fecha y hora del último cambio de estado, modificación y acceso.**
- **Un juego de permisos de acceso.**
- **Un dueño y grupo al que pertenece.**

Ejemplos:

```
-rw-r--r--   1 antonio  users         101 Dec 15  1995 new.xinitrc
-rw-r--r--   1 antonio  users        4681 Apr 20 20:02 imprecarr
```

Archivos Especiales de Dispositivo

Cada dispositivo físico en el sistema, tales como discos rígidos y flexibles, impresoras, terminales, memoria, etc. son asignados a un "archivo especial", el que a pesar de tener un tratamiento particular es visto por el sistema como un archivo común, ocupando la misma jerarquía.

Ej.:

```
brw-rw----   1 root     floppy     2,  29 Apr 27  1995 fd1H1440
brw-rw----   1 root     disk       3,   1 Apr 27  1995 hda1
crw-rw----   1 root     daemon     6,   0 Apr 27  1995 lp0
crw-rw----   1 root     kmem       1,   1 Jul 17  1994 mem
crw-rw-rw-   1 root     tty        4,  66 Jul 17  1994 ttyS2
```

Archivos de Directorio

Son lugares donde se guarda conceptual-mente, no físicamente, los archivos, los que pueden ser de cualquiera de las tres clases mencionadas. Un archivo de directorio es lo que normalmente se conoce como "directorio".

Ej.:

```
drwxr-xr-x   2 antonio  users        1024 Aug  3 19:27 nwserv
drwxrwxr-x   3 antonio  users        1024 Jul 10 00:52 tclbook
drwxr-xr-x   3 antonio  users        1024 May 21 16:10 tcltk
drwxr-xr-x   2 root     bin          2048 Apr  4 11:47 bin
```

Protección de Archivos

En todo sistema multiusuario es necesario proteger los archivos y directorios, negándose el acceso a algunos usuarios mientras se permite a otros. Los archivos son protegidos por permisos de acceso, recordar que cuando decimos archivos nos referimos a cualquiera de las tres clases.

UNIX provee tres niveles de permisos de acceso.

Permisos de Acceso

ARCHIVOS ORDINARIOS

read
Permite a un usuario ver el contenido del archivo. No se pueden guardar las modificaciones que se hallan hecho editándolo.

write
permite modificar el contenido del archivo o borrarlo.

execute
Si el archivo es un programa, el permiso de ejecución permite correrlo, caso contrario esto es impedido por el sistema.

Permisos de Acceso

ARCHIVOS DE DIRECTORIO

read

Permite ver el contenido del directorio. Esto es: se puede pedir un listado de archivos. Para esto también será necesario poseer el permiso de ejecución.

write

permite modificar el contenido de un archivo o borrarlo.

execute

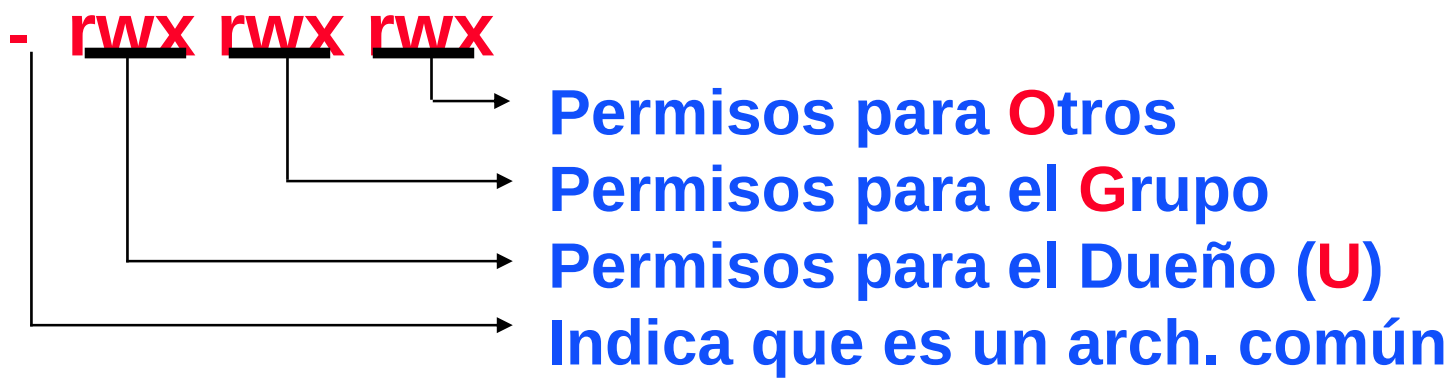
En este caso, si falta no se puede circular por el directorio en cuestión. De esta manera no se puede pedir el listado de archivos, ni utilizarlo como camino de paso hacia otro subdirectorio.

Permisos de Acceso

ENTIDADES EN UN SISTEMA UNIX

U	User	El dueño del archivo
G	Group	Grupo al que pertenece el archivo.
O	Others	Resto del mundo

Interpretación de un listado:



Propiedad de Archivos

El dueño de un archivo es quien lo creó. La propiedad se puede cambiar con el comando **chown** (change owner). El grupo al que pertenece un archivo se cambia con el comando **chgrp.**

Ej.:

```
gema:/home/antonio# ls -l apu*
-rw-r--r--   1 antonio  users        1664 Sep 27 10:52 apunte
gema:/home/antonio# chown root apunte
gema:/home/antonio# ls -l apu*
-rw-r--r--   1 root     users        1664 Sep 27 10:52 apunte
gema:/home/antonio# chgrp utn apunte
gema:/home/antonio# ls -l apu*
-rw-r--r--   1 root     utn          1664 Sep 27 10:52 apunte
gema:/home/antonio# chown antonio.users apunte
gema:/home/antonio# ls -l apu*
-rw-r--r--   1 antonio  users        1664 Sep 27 10:52 apunte
gema:/home/antonio#
```

Propiedad de Archivos

En algunos sistemas se reserva el cambio de propiedad de un archivo sólo al administrador y los usuarios comunes pueden cambiarlo de grupo sólo si pertenecen al grupo destino.

Ejemplos:

```
gema:~$ ls -l apunte
-rw-r--r--   1 antonio  users        1664 Sep 27 10:52 apunte
gema:~$ chown root apunte
chown: apunte: Operation not permitted
gema:~$ ls -l apunte
-rw-r--r--   1 antonio  users        1664 Sep 27 10:52 apunte
gema:~$ chgrp uucp apunte
chgrp: you are not a member of group `uucp': Operation not permitted
gema:~$ ls -l apunte
-rw-r--r--   1 antonio  users        1664 Sep 27 10:52 apunte
gema:~$ chgrp utn apunte
gema:~$ ls -l apunte
-rw-r--r--   1 antonio  utn          1664 Sep 27 10:52 apunte
gema:~$
```

Cambio de Permisos en Archivos

Lo realiza el dueño del archivo o el administrador del sistema mediante el comando chmod
Existen dos formas en que se puede operar:

- **Simbólica**
- **Numérica**

Forma Simbólica:

```
gema:~$ ls -l apunte
-rw-r--r--   1 antonio  users        1664 Sep 27 10:52 apunte
gema:~$ chmod o+x apunte
-rw-r--r-x   1 antonio  users        1664 Sep 27 10:52 apunte*
gema:~$ chmod o-r,g+w apunte
-rw-rw---x   1 antonio  users        1664 Sep 27 10:52 apunte*
gema:~$ chmod u+rwx apunte
-rwxrw---x   1 antonio  users        1664 Sep 27 10:52 apunte*
gema:~$ chmod u=r apunte
-r--rw---x   1 antonio  users        1664 Sep 27 10:52 apunte*
gema:~$
```

Cambio de Permisos en Archivos

Forma Numérica:

Los tres niveles (rwx) se pueden representar por dígitos octales, que tienen un peso asignado de acuerdo a su posición. A su vez esto se repite para cada una de las entidades (dueño, grupo, otros), con lo que se forma un número octal de tres dígitos.

rwx 421

4, 2 y 1 son los valores en el sistema de numeración octal que tienen los tres niveles mostrados (rwx), x= 1, w= 2 y r= 4. Si están los tres la suma de ellos da 7.

Cambio de Permisos en Archivos

Forma Numérica (continuación):

`rwx`	da 7	`rw-`	da 6
`r-x`	da 5	`r--`	da 4
`-wx`	da 3	`-w-`	da 2
`--x`	da 1	`---`	da 0

Con esto se obtiene el dígito octal de una posición, lo mismo sucede para las restantes. Por ejemplo:

-rw-r--r-- = 644

Ejemplos:

(1) `rwxrwxrwx`

(2) `rwxr-xr-x`

(3) `rw-r--r--`

(4) `r-xr-xr-x`

(5) `r--------`

(6) `rwxr-xr--`

Enlaces (links)

Con el fin de ahorrar espacio, Unix permite generar enlaces a archivos. Esto significa que al mismo archivo se lo puede ver desde distintos lugares, a partir del enlace correspondiente.

Hay dos clases de enlaces, los comunes o duros y los simbólicos.

Uso:

ln origen destino

A partir del archivo "origen" generamos un enlace "destino". A partir de ahora ambos tienen la misma categoría y no hay distinción entre ellos

Enlaces (links)

Ejemplo:

```
antonio@atsrv:~$ ls -l origen
-rwxr-xr-x  1 antonio antonio 13396 Apr  6 10:44 origen
antonio@atsrv:~$ ln origen destino
antonio@atsrv:~$ ls -l origen
-rwxr-xr-x  2 antonio antonio 13396 Apr  6 10:44 origen
antonio@atsrv:~$ ls -l destino
-rwxr-xr-x  2 antonio antonio 13396 Apr  6 10:44 destino
antonio@atsrv:~$ rm origen
antonio@atsrv:~$ ls -l destino
-rwxr-xr-x  1 antonio antonio 13396 Apr  6 10:44 destino
antonio@atsrv:~$ rm destino
antonio@atsrv:~$ ls -l origen destino
ls: origen: No such file or directory
ls: destino: No such file or directory
antonio@atsrv:~$
```

Enlaces simbólicos

Es una variante de los enlaces (links) y permite que el enlace exista aunque físicamente el archivo no se haya generado todavía.

Amplía el concepto permitiendo enlaces simbólicos a directorios.

Actualmente son muy usados, casi con mayor frecuencia que los enlaces duros.

ln -s origen destino

En este caso destino existe como enlace simbólico a origen, aunque origen todavía no exista.

Enlaces simbólicos

```
antonio@atsrv:~$ ls -l origen
ls: origen: No such file or directory
antonio@atsrv:~$ ln -s origen destino
antonio@atsrv:~$ ls -l destino
lrwxrwxrwx  1 antonio antonio 6 Apr  6 10:53 destino -> origen
antonio@atsrv:~$ cat > origen
Primera linea
segunda linea
^d
antonio@atsrv:~$ cat destino
primera linea
segunda linea
antonio@atsrv:~$ ls -l origen destino
lrwxrwxrwx  1 antonio antonio  6 Apr  6 10:53 destino -> origen
-rw-r--r--  1 antonio antonio 28 Apr  6 10:53 origen
antonio@atsrv:~$ rm origen
antonio@atsrv:~$ ls -l origen destino
ls: origen: No such file or directory
lrwxrwxrwx  1 antonio antonio 6 Apr  6 10:53 destino -> origen
```

Ingreso y Egreso al sistema

- Se accede a un equipo con sistema operativo Linux mediante el uso de un **login**.
- Cada login consiste en los siguientes componentes:

  nombre de login
  número de identificación de usuario (uid)
  número de identificación de grupo (gid)
  contraseña (password)

Ingreso al sistema

- Estos atributos están guardados en el archivo: /etc/passwd

```
cat /etc/passwd
```

```
postmaster:x:14:12:postmaster:/var/spool/mail:/bin/bash
antonio:x:501:100:Antonio Trevi#o:/home/antonio:/bin/bash
jorgek:x:502:100:Jorge Kremer:/home/jorgek:/bin/bash
gopher:x:503:100:Usuario de gopher:/var/httpd/htdocs:
uuastro:x:504:100:Correo desde astro:/var/spool/uucp:/usr/lib/uucico
```

Un nombre de login, o simplemente login, es una cadena única de letras y números que identifican un individuo en el sistema. El login debe empezar con una letra. En muchos casos el login de un persona es su primer nombre, apellido, iniciales o sobrenombre.

Ingreso al sistema

- El uid de un login es un número único asignado a cada cuenta de usuario por el administrador.
- El gid es un número único asignado a cada grupo. Esto identifica un conjunto de usuarios que tienen algo en común. El gid es importante para la seguridad del sistema.

Sesión Típica:

```
login as: antonio
Using keyboard-interactive authentication.
Password:
Linux atsrv 2.4.27-2-386 #1 Tue Aug 16 15:28:25 JST 2005 i686 GNU/Linux

Last login: Tue May 23 00:15:00 2006 from jl.synwave.com.ar
antonio@atsrv:~$
```

Ingreso al sistema

- Se puede hacer el ingreso a partir de un entorno gráfico con X Window.
- Una máquina con Linux tiene un conjunto de terminales (el número es modificable) a las que se accede mediante ALT-F1 hasta ALT-Fx, siendo x la última disponible.
- El entorno gráfico generalmente es la terminal número 7.

Vías de Ingreso al sistema

- Es posible ingresar al sistema desde cualquier terminal sobre la consola (por defecto tenemos 6 de texto y 1 gráfica).
- A lo anterior se adicionan las seudoterminales por red, a las que se accede mediante el protocolo telnet desde cualquier sistema operativo con TCP/IP.
- También se puede ingresar a partir de un servidor X Window desde cualquier sistema operativo que lo soporte.

Egreso del sistema

- Cuando no se va a trabajar más en una terminal, aunque sea en forma temporaria, hay que salir formalmente del sistema.
- Para esto hay que realizar un logout, originalmente se hacía con <CTRL-D> que significa fin de archivo, pero ahora se hace escribiendo: logout <Enter>
  ó: exit <Enter>

Ejemplo:

```
antonio@atsrv:~$ exit
Bienvenido a atsrv - 2.4.27-2
atsrv login:
```

Egreso del sistema

- Si se está usando un entorno gráfico (X Window), debe verificarse si el ingreso se produjo por esa vía o desde una terminal de texto con la que se lanzó.
- En el primer caso la salida de la terminal gráfica implicará salida del sistema.
- En el segundo debe verificarse que aparte del entorno gráfico cerremos la terminal que lo lanzó.

Comandos del Sistema Operativo

ls Muestra el contenido de un directorio

Sintaxis:

```
ls [-opciones] [directorio|archivo]
```

Opciones más importantes:

- **-l** Información completa de los archivos.
- **-x** Lista en columnas, ordenamiento horizontal.
- **-C** Lista en columnas, ordenamiento vertical.
- **-d** En directorios lista el nombre, no el contenido
- **-R** Lista directorios en forma recursiva
- **-a** Lista todas las entradas, incluyendo las ocultas

Comandos del Sistema Operativo

ls (Continuación)

Opciones más importantes:

`-t`	Ordena el listado por fecha de última modificac.
`-u`	Ordena el listado por fecha de último acceso
`-c`	Ordena el listado por fecha de último cambio de estado.
`-r`	Ordena listados en forma inversa.
`-i`	Muestra los nodos i

Comandos del Sistema Operativo

cat Copia el contenido de un archivo en otro (incluyendo pantalla y teclado)

Sintaxis: `cat archivo...`

Ejemplos:

```
(a) cat notas
(b) cat > notas
(c) cat >> notas
(d) cat nota1 nota2 > notas

(a) muestra contenido del archivo "notas"
(b) crea y escribe en el archivo "notas"
(c) agrega texto al final del archivo "notas"
(d) copia nota1 y nota2 en notas
```

Comandos del Sistema Operativo

more Muestra el contenido de uno o más archivos en forma paginada.

Sintaxis: `more archivo...`

Acciones dentro del more:

<espacio>	Avanza una pantalla
<RETURN>	Avanza una línea
/texto	Busca la primera ocurrencia de texto".
h	Ayuda de comandos
!	Ejecuta un comando, regresando al more cuando finaliza.
=	Muestra el número de línea sobre la que está el puntero.
v	Ingresa al editor "vi"
q	Termina y regresa al Sistema Operativo.

Comandos del Sistema Operativo

less Muestra el contenido de uno o más archivos en forma paginada.

Sintaxis: `less archivo...`

less es similar a more

las diferencias son:

- less permite retroceder al visualizar el archivo.
- less no necesita leer todo el archivo antes de poder visualizarlo.
- less se adapta mejor que more a las condiciones particulares de un entorno.
- Las acciones son iguales a las de more.

b `Retrocede una pantalla.`

Comandos del Sistema Operativo

cp Copia archivos a otros archivos o a directorios.

Sintaxis:

```
cp arch_origen arch_destino
cp archivo directorio
```

Opciones más importantes:

- **-i** Pregunta antes de sobreescribir archivos.
- **-p** Preserve el dueño, grupo, permisos y fecha.
- **-R** Copia directorios en forma recursiva.

Comandos del Sistema Operativo

mv Renombra archivos o directorios, mueve archivos de un lugar a otro. Mover es copiar y después borrar el original.

Sintaxis:

```
mv nombreviejo nombrenuevo
mv origen destino
```

cd Cambio de directorio

Sintaxis:

```
cd [directorio]
```

Si se usa "cd" sólo, el cambio es al home directory.

Comandos del Sistema Operativo

rm Borra archivos de un directorio.

Sintaxis:

```
rm [opciones] nombrearchivo
```

Opciones más importantes:

- **-i** Pregunta en forma interactiva por cada archivo que debe borrar.
- **-r** Borra en forma recursiva todo el directorio indicado y los subdirectorios. ¡Usarlo con mucho cuidado!

Comandos del Sistema Operativo

pwd Muestra el directorio de trabajo.

Sintaxis:

```
pwd
```

head Muestra las primeras líneas de un archivo.

Sintaxis:

```
head [-cant_líneas] archivo...
```

La cantidad de líneas a mostrar está dada por el valor "cant_líneas", por defecto es: 10.

Comandos del Sistema Operativo

tail Similar a head, con el mismo parámetro pero para la última parte del archivo.

Sintaxis:

```
tail [-cant_líneas] archivo...
tail -f archivo...
```

La opción "-f" es muy utilizada. Cuando llega al final del archivo se detiene. Si el archivo crece, muestra las nuevas líneas. Su principal uso es en control de archivos de log, cuando se intenta depurar algo.

Comandos del Sistema Operativo

mkdir Crea uno o más directorios

Sintaxis:

```
mkdir [opciones] dir1 [dir2 dir3 ... dirn]
```

Opciones más importantes:

- **-p** Permite crear una cadena de directorios completa.
- **-m** Permite especificar los permisos de nuevo directorio. (Ver comando chown).

Comandos del Sistema Operativo

rmdir Borra uno o más directorios, éstos deben estar vacíos para ser borrados.

Sintaxis:

```
rmdir dir1 [dir2 dir3 ... dirn]
```

Opciones más importantes:

- **-p** Permite borrar una cadena de directorios completa, siempre que los directorios estén vacíos.

Comandos del Sistema Operativo

wc Cuenta líneas, palabras y caracteres en los archivos nombrados ó en la entrada estándar si no se da nombre de archivo.

Sintaxis:

```
wc [opciones] [nombres]
```

Opciones más importantes:

- **-l** Cuenta sólo la cantidad de líneas.
- **-w** Cuenta sólo la cantidad de palabras.
- **-c** Cuenta sólo la cantidad de caracteres.

Comandos del Sistema Operativo

find Busca archivos en el filesystem que concuerden con un criterio de selección, ejecutando sobre ellos una acción especificada. find desciende recursivamente por el árbol de directorios.

Sintaxis:

```
find camino(s) expresión
```

Opciones más importantes:

-name nomarch

Es verdadero si el patrón "nomarch" coincide con el nombre del archivo encontrado.

Comandos del Sistema Operativo

find (Continuación) Opciones importantes:

`-links n`

Indica el número de links que debe tener el archivo.

`-atime n`

Es verdadera si el archivo ha sido accedido hace más de n días.

`-mtime n`

Es verdadera si el archivo ha sido modificado hace más de n días.

`-user nombre`

Es verdadera si el dueño del archivo encontrado coincide con "nombre".

Comandos del Sistema Operativo

find (Continuación) Opciones importantes:

`-group nombre`

Es verdadera si el grupo a que corresponde el archivo encontrado coincide con "nombre"

`-size n[c]`

Es verdadera si el archivo tiene una longitud n de bloques. Si el valor n está seguido por una "c", la indicación es en caracteres.

`-print`

Si está es siempre verdadera y hace que se imprima el camino del archivo localizado.

Comandos del Sistema Operativo

find (Continuación)

Opciones más importantes:

`-exec comando []\;`

Ejecuta el comando "comando" sobre los archivos encontrados, los "{}" representan el nombre del archivo (son requeridos por la sintaxis) y el ";" (también requerido) es una separación necesaria de las sucesivas ejecuciones de comandos generadas por cada archivo encontrado.

`-perm -num`

Verdadero si el archivo encontrado tiene las permisiones iguales a "num", num es un número octal y es indicación absoluta.

Comandos del Sistema Operativo

find (Continuación)

Ejemplos:

```
find / -name planilla -print
```

Es el uso más común del find y lo que hace es imprimir el directorio donde se encuentra el archivo "planilla". La barra "/" indica que la búsqueda comienza a partir de la raíz.

```
find . -name planilla -print
```

Igual que antes, pero la búsqueda comienza en el directorio que estamos trabajando.

Comandos del Sistema Operativo

find (Continuación)

Ejemplos:

```
find / -name core -exec rm {}\;
```

Comienza la búsqueda de los archivos "core" a partir de la raíz y los borra.

```
find / -perm -777 -print
```

A partir de la raíz busca todos los archivos que tengan permisión 777 e imprime su nombre en pantalla. Es útil para saber qué archivos tienen bajos niveles de seguridad.

Comandos del Sistema Operativo

grep Busca un patrón en uno o más archivos, ejecutando las acciones indicadas por las opciones. Si no se especifica acción simplemente imprime las líneas que contienen el patrón.

Sintaxis:

```
grep [opciones] expresión archivo(s)
```

Opciones más importantes:

-v Muestra todas las líneas excepto las que contienen el patrón.

Comandos del Sistema Operativo

grep (Continuación)

Opciones más importantes:

- **-c** Cuenta las líneas que contienen el patrón.
- **-l** Muestra solo los nombres de los archivos que contienen el patrón, uno en cada línea.
- **-n** La impresión de las líneas que contienen el patrón es precedida por el número de línea en el archivo.
- **-b** La impresión de las líneas que contienen el patrón es precedida por el desplazamiento de la línea en bytes desde el comienzo del archivo.

Comandos del Sistema Operativo

grep (Continuación)

Opciones más importantes:

`-s` Suprime los mensajes de error producidos por la no existencia o imposibilidad de leer un archivo. Util cuando se especifican varios.

`-i` Hace que el patrón coincida independientemente si está en mayúsculas o minúsculas.

Ejemplos: `grep carga mens*`

Busca el patrón "carga" en todos los archivos cuyo nombre comienza con "mens" en el directorio de trabajo, e imprime las líneas que lo contienen.

Comandos del Sistema Operativo

grep (Continuación)

Ejemplos: `grep '[Ff]ono' agenda`

Imprime todas las líneas en el archivo "agenda" que contienen el patrón Fono ó fono.

`grep -i fono agenda`

Imprime todas las líneas en el archivo "agenda" que contienen el patrón fono, sin interesar si está con mayúsculas o minúsculas. Observar que esto no es lo mismo que el ejemplo anterior. En aquel solo se permite que la primera letra sea "F" ó "f", el resto deben ser minúsculas.

Comandos del Sistema Operativo

df Muestra un resumen de la ocupación de disco de los volúmenes montados.

Sintaxis: `df`

Ejemplo:

```
gema:~$ df
Filesystem           1024-blocks  Used Available Capacity Mounted on
/dev/hda2               665660  457355   172773      73%   /
/dev/hda1               102566   96350     6216      94%   /dos
gema:~$
```

Sesión de Trabajo

logname Muestra el nombre con el que el usuario ingresó al sistema.

Sintaxis: `logname`

who Muestra quiénes están trabajando en el sistema en un momento determinado. Aparte es posible conocer la terminal desde la que están trabajando, hora de login y cuánto tiempo lleva esa terminal inactiva.

Sintaxis: `who [opciones]`

Sesión de Trabajo

who (Continuación)

Opciones:

-u Esta opción lista sólo los usuarios presentes, NAME: es el login de usuario. LINE: es el nombre de la línea de entrada tal como aparece en el directorio /dev. TIME: es la hora en que entró el usuario. IDLE: es el tiempo en horas y minutos de inactividad registrada en la línea "line", un punto "." en este campo indica que la terminal ha tenido actividad en el último minuto, si ha transcurrido más de 24 horas desde la última actividad aparece la palabra "old". Este campo es útil cuando se quiere saber si una persona está trabajando en la terminal o no.

Sesión de Trabajo

who (Continuación)

Opciones:

-T Es similar a -u excepto que es mostrado el estado de la terminal. state: describe si alguna otra persona puede escribir sobre esa terminal, si aparece un "+" se permite, caso contrario aparece un "-". Si hay problemas con la línea, aparece un "?". root puede escribir a cualquier terminal, independientemente de su estado.

-H Esta opción muestra los encabezamientos de las columnas.

Sesión de Trabajo

who (Continuación)

Opciones:

-q Opción rápida, mustra sólo los nombres y la cantidad de usuarios que han registrado su ingreso en el sistema. Esta opción excluye todas las demás.

-m Muestra solo la información relativa a quien convoca el comando, es equivalente a:

```
who am i
```

Sesión de Trabajo

whoami — **Permite visualizar bajo qué login se está trabajando.**

Para interrumpir un comando en ejecución se usa:

<CRTL> c

Pulsar la "c" mientras se tiene presionada la tecla Control.

Sesión de Trabajo

tty Muestra el camino (pathname) de la terminal en el sistema como dispositivo.

Sintaxis: `tty`

Ejemplo:
```
tty
/dev/tty1
```

Indica que estamos trabajando en la primera pantalla del sistema.

clear Limpia la pantalla

Sintaxis: `clear`

Sesión de Trabajo

write Sirve para comunicación con otro usuario. Es por líneas completas.

Sintaxis: `write usuario [tty]`

Ejemplo: `write jose`

En la pantalla de Juan aparece:

`Message from juan@nombresistema on ttyp2 at 10:45...`

Se contesta escribiendo:

`write juan`

Es conveniente establecer una bandera de intercambio.

Sesión de Trabajo

mesg Habilita o deshabilita la terminal para recibir mensajes.

Sintaxis: `mesg [y | n]`

El argumento "n" impide que otros usuarios hagan write a nuestra terminal.
Lo que hace en realidad es modificar la permisión "w" para la entidad "otros" en el archivo especial de dispositivo ubicado en el directorio /dev.
El argumento "y" permite que otros hagan write sobre nuestra terminal.

mesg sin argumentos informa del estado de nuestra terminal en ese aspecto.

Sesión de Trabajo

talk Comunicación interactiva con otro usuario a pantalla partida.

Sintaxis: `talk usuario@host [tty]`

wall Este comando sirve para enviar mensajes a todos los usuarios,

Sintaxis: `wall mensaje`

date Para los usuarios comunes sirve para ver la fecha y hora actual, el supervisor además tiene el derecho de modificarlas.

sintaxis: `date`

Sesión de Trabajo

cal Imprime un calendario para el año y mes especificado.

Sintaxis: `cal[[mes] año]`

Si no se suministran argumentos la salida es el mes actual del corriente año.
Si se especifica sólo el año éste se imprime completo.
Si se indica sólo un mes se interpreta como del corriente año. El año debe estar entre 1 y 9999, el mes entre 1 y 12 o los caracteres suficientes (en inglés) que diferencien un mes de otro. Tener en cuenta que el año debe indicarse con cuatro cifras, 93 es el año noventa y tres, no 1993.

Sesión de Trabajo

uname Imprime información sobre el sistema.

Sintaxis: `uname [-a]`

Dispone de varias opciones, todas resumidas con (-a), la que indica el tipo y versión del sistema operativo, fecha y hora y tipo del procesador sobre el que está corriendo.

passwd Permite cambiar el

Sintaxis: `passwd`

El texto ingresado no es visible y es solicitado dos veces para asegurarse que fue ingresado correcta-mente.

Sesión de Trabajo

ps Lista los procesos que están activos al momento de ejecutar el comando.

Sintaxis: `ps [-opciones]`

Opciones:

- **-a** Lista todos los procesos (no solo los propios)
- **-f** Muestra las dependencias de procesos.
- **-x** Muestra los procesos sin terminal de control
- **-u** Muestra el nombre de usuario que inició el proceso, su hora de comienzo y otros datos.

Nota: ps es un comando en transición, en algunos sistemas las opciones funcionan sin guiones.

Almacenamiento de Información

tar Tape archiver, sirve para almacenar información en un archivo, tratándolo como si fuera una cinta. Guarda un árbol de directorios en un archivo.

Sintaxis: `tar cvf dispositivo|archivo nomarch`

Grabar desde el principio

`tar rvf dispositivo|archivo nomarch`

Grabar a continuación del último.

`tar tvf dispositivo|archivo nomarch`

Ver el contenido de cinta o disquete.

`tar xvf dispositivo|archivo nomarch`

Extracción de archivos.

Almacenamiento de Información

tar Compresión de Archivos.

Para todos los casos anteriores, excepto “r” (agregado), se puede especificar compresión del archivo tar, ej:

```
tar zcvf dispositivo|archivo nomarch.tgz
```

“nomarch.tgz” estará comprimido por el utilitario gzip.

```
tar zxvf nomarch.tgz
```

Extraerá componentes de un archivo comprimido.

Ordenamiento de Contenidos

sort Ordena líneas de archivos de texto.

Por defecto escribe el resultado en la salida estándar.

Sintaxis: `sort nomarch`

Opciones:

- **-r** Ordenamiento inverso.
- **-f** Ignora diferencia mayúsculas/minúsculas.
- **-g** Ordenamiento numérico.
- **-o** Escritura de la salida en archivo cuyo nombre va a continuación.

Diferencia de Contenidos

diff Compara archivos de texto línea por línea.

Escribe el resultado en la salida estándar.

Sintaxis: `diff arch1 arch2`

Opciones:

- `-b` Ignora cambio de espacios en blanco
- `-i` Ignora diferencia mayúsculas/minúsculas.
- `-w` Ignora todos los espacios.
- `-B` Ignora líneas en blanco

Ayuda de comandos

- Las páginas de manual de un sistema Unix están casi siempre en línea. Para conocer el uso de un comando (ej. "cp") hacemos:

  `man cp`

- Si no se está seguro del nombre del comando resulta muy útil apropos. Ej. con "copy":

  `apropos copy`

- Saber dónde se encuentra un comando es útil en muchas oportunidades, se hace con:

  `which cp`

NAME

ls - list directory contents

SYNOPSIS

ls [OPTION]... [FILE]...

DESCRIPTION

List information about the FILEs (the current directory by default). Sort entries alphabetically if none of **-cftuSUX** nor **--sort**.

Mandatory arguments to long options are mandatory for short options too.

-a, **--all**
do not ignore entries starting with .

-A, **--almost-all**
do not list implied . and ..

--author
with **-l**, print the author of each file

-b, **--escape**
print octal escapes for nongraphic characters

Ayuda de comandos

- Para una rápida descripción de un comando usamos whatis. Ejemplo, queremos saber de qué se trata el comando cp

```
whatis cp
cp (1)              - copy files and directories
```

- Para conocer la ubicación de un comando y su página de comandos

```
whereis cp
cp: /bin/cp /usr/share/man/man1/cp.1.gz
```

Ejecución Diferida de comandos

UNIX dispone de un mecanismo que permite ejecutar tareas en tiempo diferido, básicamente tiene tres posibilidades que cubren aspectos diferentes:

- crontab
- at
- batch

crontab ejecuta actividades periódicamente en días y horarios determinados. cron es el nombre del programa que soporta esta modalidad de funcionamiento y crontab es el comando para utilizarlo.

Ejecución Diferida de comandos

Crontab (continuación)

Sintaxis:

```
crontab archcron
crontab -e
crontab -r
crontab -l
```

El archivo de crontab de cada usuario está en el directorio `/usr/spool/cron/crontabs` con el nombre del usuario.

`crontab archcron`

Copia el archivo "archcron" en el directorio que contiene los crontab de todos los usuarios y se ejecuta.

Ejecución Diferida de comandos

Crontab (continuación)

`crontab -e`

Edita el archivo de crontab del usuario que invoca el comando con el editor por defecto.

`crontab -r`

Remueve del directorio de crontabs el archivo de crontab del usuario que invoca el comando.

`crontab -l`

Lista el contenido del archivo correspondiente al usuario que invoca el comando en el directorio de crontabs.

Ejecución Diferida de comandos

at Difiere la ejecución de un comando o conjunto de comandos para un día y horario determinado.

Sintaxis:

```
at hora
primer comando
   ...
   ...
último comando
CTRL-d
```

Hora debe reemplazarse por la hora real que se pretende que corra el comando. Además de la hora se puede especificar un día y mes, esto último es opcional.

Ejecución Diferida de comandos

at (Continuación)

Ejemplo:

```
at 11:45am Jul 12
mail arturo < reporte
CTRL-d
```

El comando at responde con el número de trabajo (job number) y que será ejecutado utilzando el /bin/sh.

Opciones:

```
at -d jobnumber        atrm jobnumber
at -l                  atq
at -m
```

Ejecución Diferida de comandos

at (Continuación)

`at -d jobnumber` `atrm jobnumber`

Se usa para remover un comando que está indicado para ejecución diferida, jobnumber es el número de trabajo que dió el at al encargar la ejecución diferida. Si no se lo recuerda, se puede averiguar con la opción siguiente.

`at -l` `atq`

Sirve para listar los trabajos pendientes de ejecución con at y con batch.

`at -m`

Hace que se envíe mail al usuario cuando haya finalizado la ejecución del trabajo.

Ejecución Diferida de comandos

Formato de archivo para crontab

Cada línea del archivo define un procedimiento a realizarse a un hora determinada.

```
minutos hora día mes día-de-la-semana comando [parms]
```

Los campos se define de la forma siguiente:

```
minutos          (0-59)
hora             (0-23)
día              (1-31)
mes              (1-12)
día-de-la-semana (0-6 con 0=domingo)
comando          (el comando a ser ejecutado)
[parámetros] lista de parámetros opcionales.
```

Ejecución Diferida de comandos

Ejemplos de archivos de crontab

Un asterisco en un campo implica todos los valores posibles.

```
0 8 1-5,10,15 * * mail jorge antonio < estado
```

Indica que a la hora 08:00 durante los días 1 al 5, 10 y 15 de cada mes enviará el archivo estado como mail a los usuarios jorge y antonio.

```
15 11,23 * * 1-5 rm /tmp/* /usr/tmp/*
```

Indica que de lunes a viernes a las 11:15 y 23:15 de todos los meses (en forma continua), borrará los directorios /tmp y /usr/tmp.

Ejecución Diferida de comandos

Ejemplos:

A continuación vemos un ejemplo de una línea de crontab que se ejecutará una vez al año, exactamente el día que queremos enviar felicitaciones a alguien por su cumpleaños. Primero, asumiendo que nuestro nombre de usuario es juan y que nuestro home directory es /home/juan, escribimos un archivo que llamamos p. ej. ”cumple.rc” con el siguiente contenido:

```
echo ”Felicidades...” > /tmp/a
echo ”que tengas un muy buen dia.” >> /tmp/a
mail -s ”Que los cumplas MUY FELIZ, $1!!!” $1 < /tmp/a
rm /tmp/a
```

el que hacemos ejecutable.

Ejecución Diferida de comandos

Ejemplos (Continuación del anterior)

En un segundo archivo, que usaremos para el crontab, llamado p. ej.: ”cumple.rc”’escribimos lo siguiente:

```
0 8 29 6 * /home/juan/cumple antonio
```

Asumiendo que el destinatario es antonio.

Por último hacemos:

```
crontab cumple.rc
```

y mientras no borremos el archivo ”cumple.rc”, se ejecutará en la fecha indicada lo siguiente:

Ejecución Diferida de comandos

Ejemplos (Conclusión)

El comando enviará un mail a antonio el 29 de Junio de cada año a las 8 de la mañana en punto, con el Subject: "Que los cumplas MUY FELIZ, antonio!!! " y el cuerpo del mail será el generado por los comandos contenidos en archivo cumple, esto es: la generación del archivo "/tmp/a", reemplazando el parámetro "$1" por "antonio", por último borrará el archivo temporario "/tmp/a" que utilizó para enviar el mail.

El Shell

- Cuando los usuarios se comunican con el sistema, lo hacen con un intérprete de comandos: **shell**.
- Los sistemas Unix poseen varios shells.
- El usuario puede elegir el que le resulta más conveniente.
- Los shells no sólo son intérpretes de comandos sino que cumplen las funciones de lenguajes de programación.

El Shell

- Básicamente en Unix hay dos clases de ejecutables, los binarios y los scripts que son archivos con comandos usando el shell.

Ej: `cat uno dos tres`

cat es un ejecutable (comando del sistema), al que se le pasan tres parámetros. cat es el parámetro 0; uno, dos y tres son respectivamente los parámetros 1, 2, 3.

Los ejecutables se cargan en memoria directamente, para ejecutar un script el shell invoca una copia de si mismo (proceso hijo) que interpretará los comandos del script.

Metacaracteres

- Los metacaracteres son caracteres con significado especial para el shell, dependiendo del contexto. No hay que usarlos cuando se está dando nombre a un archivo ya que podemos tener complicaciones al referirnos a él en el futuro.

Metacaracteres en Unix:

`*  ?  []  '  ` " / \ | > < >> $`

`*` Reemplaza 0 o más caracteres.

`?` Reemplaza 1 solo caracter.

`''` Se interpreta literalmente los que está en su interior.

Metacaracteres

``Lo que está adentro debe interpretarse como un comando.

""Lo que está adentro es texto literal, pero deben interpretarse los metacaracteres.

/ Barra divisora de directorios.

\ El carácter que sigue debe interpretarse literal-mente.

| Barra encadenadora del mecanismo del pipe.

> Dirige la salida hacia.

< Toma la entrada desde.

>>Dirige la salida hacia agregando a lo ya existente.

$ Se usa para mostrar el contenido de la variable que sigue.

El editor SED

- SED significa Stream Editor.
- Es un editor de flujo derivado del ED, que a su vez es antecesor del Vi
- Su misión es abrir un archivo o la entrada estándar si no se especifica un nombre, leerlo línea por línea, produciendo modificaciones y escribiendo el resultado en la salida estándar, la que generalmente es redireccionada a otro archivo.
- Los comandos que el sed va a aplicar pueden provenir de un archivo.

El editor SED

- **Uso:**

  sed [-an] comando [archivo ...]
  sed [-an] -e comando [-f arch_de_comando]
  [archivo ...]

- **Opciones:**

-a **Por defecto los archivos de salida se crean no bien empieza a trabajar el SED, con esta opción se abren sólo si hay salida genera-da**

El editor SED

- **Opciones:**

-n Por defecto todo lo que se lee a la entrada aparece a la salida, esta opción suprime ese comportamiento.

-e comando. Agrega a la lista de comandos, los que aparecen bajo esta opción.

-f arch_de_comando. Toma los comandos a aplicar desde el archivo: arch_de_comando.

El editor SED

El formato de los comandos es el siguiente:

[dirección[,dirección]]función[argumentos]

- **Las direcciones especifican el inicio y el fin de la acción.**
- **Función indica la operación a realizar.**
- **Argumentos son los parámetros pasados a las funciones.**

El editor SED

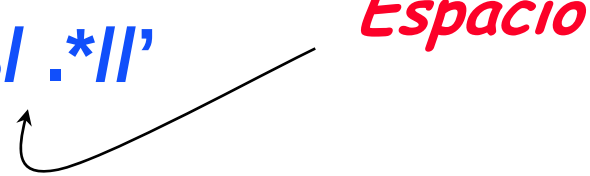
Ejemplo:

- **El comando who, por defecto tiene la siguiente salida:**

```
root     tty1     Aug 28  10:12
antonio  tty5     Aug 28  09:34
antonio  tty5     Aug 28  10:03
```

- **Si queremos ver sólo el primer campo aplicamos lo siguiente:**

who | sed 's/ .*//'



El editor SED

- **Si queremos ver el último campo. Aplicamos:**

  who | sed 's/^.* //'

- **Si queremos escribir sólo una determinada cantidad de líneas desde el principio, tenemos:**

  who | sed 'nq'

Siendo "n" la cantidad de líneas.

El editor SED

- También podemos hacer que imprima un número de líneas hasta encontrar un patrón:

  who | sed '/root/q'

- Para cancelar todas las línas que contienen el patrón tenemos:

  who | sed '/root/d'

El editor SED

- **Para inhibir el comportamiento por defecto de imprimir en la salida lo que recibe de la entrada, tenemos:**

```
who | sed -n '/root/p'
```

- **Este comando se comporta como el grep.**
- **Si quiero imprimir el archivo de entrada a doble espacio:**

```
who | sed -n 's/$/\
/'
```

El editor SED

- **Ejemplos**

sed -n ’20,30p’	**Imprime líneas de la 20 a 30**
sed ’1,10d’	**Borra las líneas de 1 a 10**
sed ’1,/^$/d’	**Borra hasta la primera línea en blanco, inclusive.**
sed -n ’/^$/,/end/p’	**Imprime líneas desde la primera vacía hasta la que tiene ”end” al comiezo.**
sed ’$d’	**Imprime todas excepto la última.**